Introduction

Computer Architecture

References & Grading

- **References**
  - David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)

References & Grading

- **References**
  - David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)
  - *Introduction to Computing Systems: From Bits and Gates to C and Beyond by* Patt & Patel

References & Grading

- **References**
  - David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)
  - *Introduction to Computing Systems: From Bits and Gates to C and Beyond by* Patt & Patel
- **Grading**
  - Midterm Exam: 46%
  - Final Exam: 50%

References & Grading

- **References**
  - David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)
  - *Introduction to Computing Systems: From Bits and Gates to C and Beyond by* Patt & Patel
- **Grading**
  - Midterm Exam: 46%
  - Final Exam: 50%
  - Homeworks: 4%

References & Grading

- **References**
  - David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)
  - *Introduction to Computing Systems: From Bits and Gates to C and Beyond by* Patt & Patel
- **Grading**
  - Midterm Exam: 46%
  - Final Exam: 50%
  - Homeworks: 4%
  - Attendance: Fail if absent more than allowed

Readings

- David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)
  - Chapter 1

A Note on Hardware vs. Software

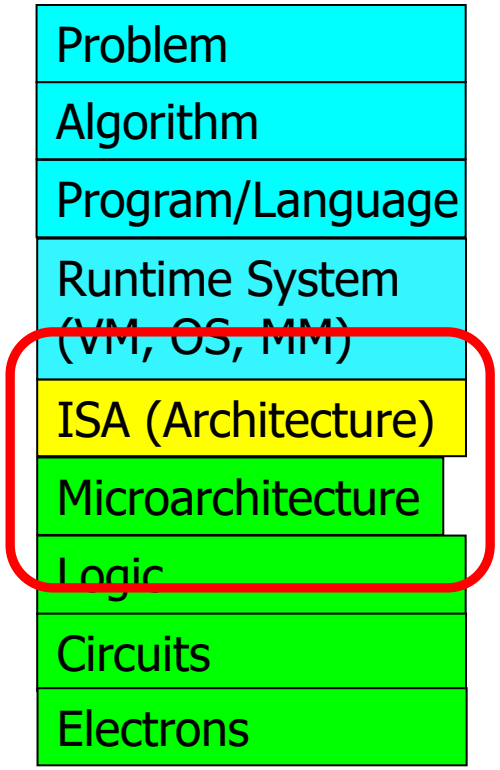
- This course is classified under “Computer Hardware”

- However, you will be much more capable if you master both hardware and software (and the interface between them)
  - Can develop better software if you understand the underlying hardware
  - Can design better hardware if you understand what software it will execute
  - Can design a better computing system if you understand both

Definition of computer architecture

- The science and art of designing, selecting, and interconnecting hardware components and designing the hardware/software interface to create a computing system that meets functional, performance, energy consumption, cost, and other specific goals.

Computer Architecture in Levels of Transformation



Aside: What Is An Algorithm?

- Step-by-step procedure where each step has three properties:
  - Definite (precisely defined)
  - Effectively computable (by a computer)
  - Terminates

The Power of Abstraction

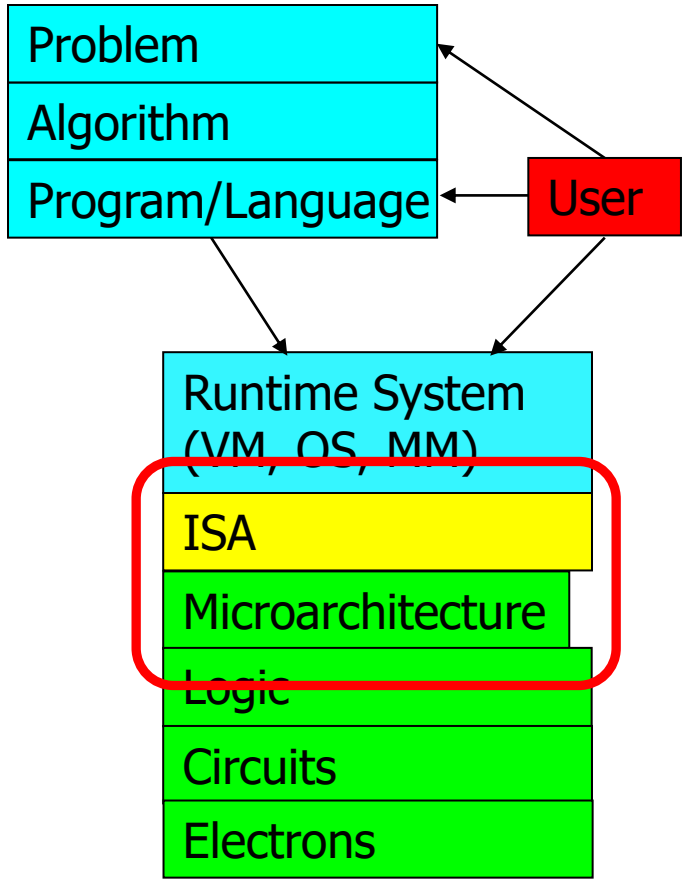
- Levels of transformation create abstractions
  - Abstraction: A higher level only needs to know about the interface to the lower level, not how the lower level is implemented
  - E.g., high-level language programmer does not really need to know what the ISA is and how a computer executes instructions
- Abstraction improves productivity
  - No need to worry about decisions made in underlying levels
  - E.g., programming in Java vs. C vs. assembly vs. binary vs. by specifying control signals of each transistor every cycle
- Then, why would you want to know what goes on underneath or above?

Crossing the Abstraction Layers

- As long as everything goes well, not knowing what happens in the underlying level (or above) is not a problem.
- What if
  - The program you wrote is running slow?
  - The program you wrote does not run correctly?
  - The program you wrote consumes too much energy?
- What if
  - The hardware you designed is too hard to program?
  - The hardware you designed is too slow because it does not provide the right primitives to the software?
- What if
  - You want to design a much more efficient and higher performance system?

Levels of Transformation, Revisited

- A user-centric view: computer designed for users



- The entire stack should be optimized for user

Course Goals

- Goal 1: To familiarize those interested in computer system design with both fundamental operation principles and design tradeoffs of processor, memory, and platform architectures in today's systems.
  - Strong emphasis on fundamentals and design tradeoffs.

- Goal 2: To provide the necessary background and experience to design, implement, and evaluate a modern processor.
  - Strong emphasis on functionality and design.

Role of The (Computer) Architect

- Look backward (to the past)
  - Understand tradeoffs and designs, upsides/downsides, past workloads. Analyze and evaluate the past.
- Look forward (to the future)
  - Be the dreamer and create new designs. Listen to dreamers.
  - Push the state of the art. Evaluate new design choices.
- Look up (towards problems in the computing stack)
  - Understand important problems and their nature.
  - Develop architectures and ideas to solve important problems.
- Look down (towards device/circuit technology)
  - Understand the capabilities of the underlying technology.
  - Predict and adapt to the future of technology (you are designing for N years ahead). Enable the future technology.

Why Study Computer Architecture?

Why Study Computer Architecture?

- Enable better systems: make computers faster, cheaper, smaller, more reliable, …
  - By exploiting advances and changes in underlying technology/circuits
- Enable new applications
  - Life-like 3D visualization 20 years ago?
  - Virtual reality?
  - Personalized genomics? Personalized medicine?
- Enable better solutions to problems
  - Software innovation is built into trends and changes in computer architecture
    - > 50% performance improvement per year has enabled this innovation
- Understand why computers work the way they do

Computer Architecture Today (I)

- Today is a very exciting time to study computer architecture

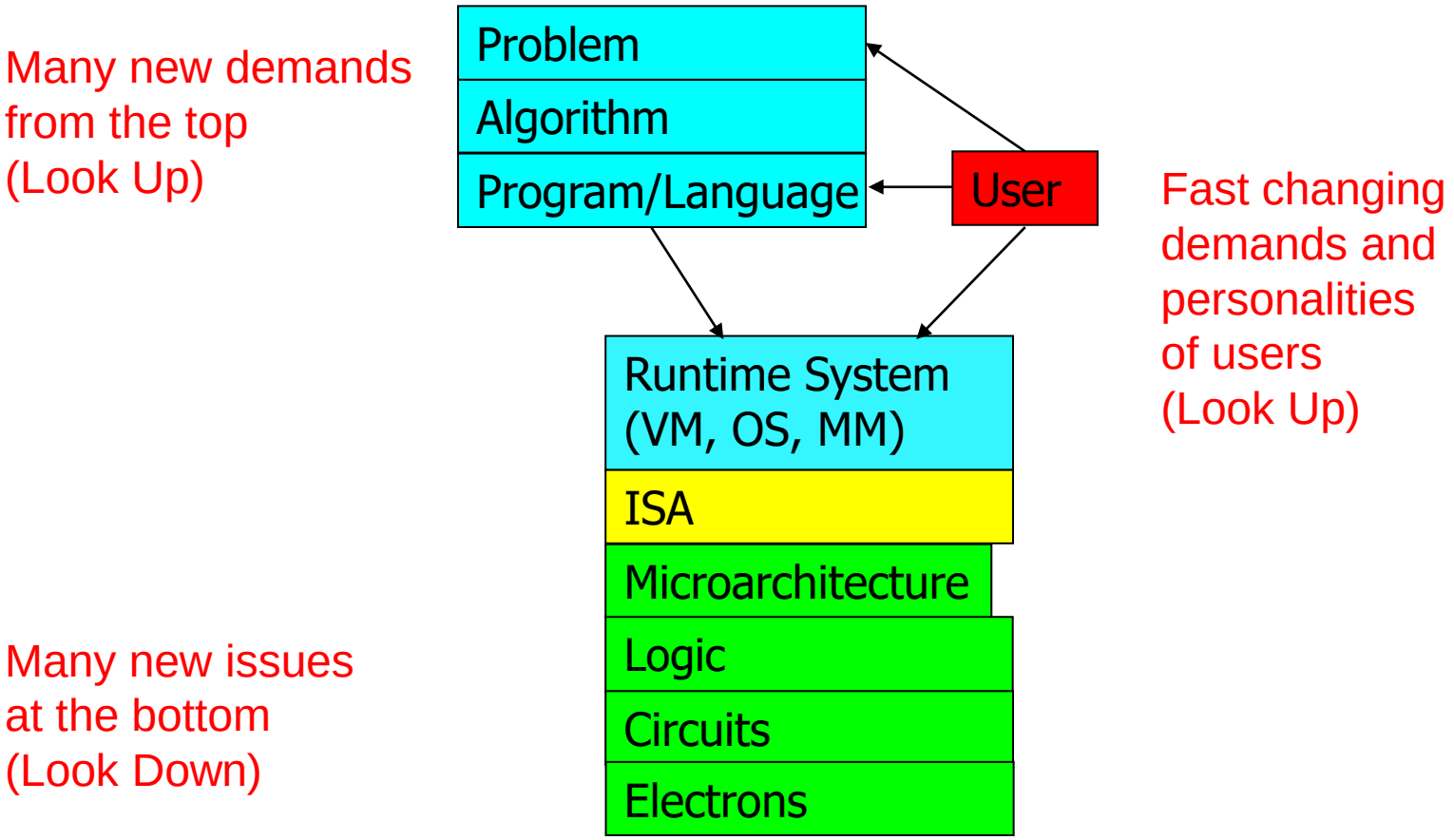
- Industry is in a large paradigm shift (to multi-core and beyond) – many different potential system designs possible

- Many difficult problems *motivating* and *caused by* the shift
  - Power/energy constraints → multi-core?
  - Complexity of design → multi-core?
  - Difficulties in technology scaling → new technologies?
  - Memory wall/gap
  - Reliability wall/issues
  - Programmability wall/problem
  - Huge hunger for data and new data-intensive applications

- No clear, definitive answers to these problems

Computer Architecture Today (II)

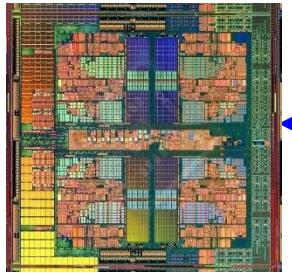
- These problems affect all parts of the computing stack – if we do not change the way we design systems



- No clear, definitive answers to these problems

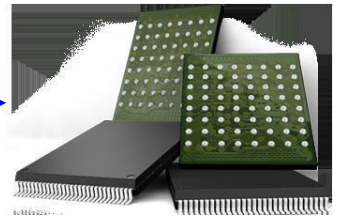
Computer Architecture Today (III)

- Computing landscape is very different from 10-20 years ago
- Both UP (software and humanity trends) and DOWN (technologies and their issues), FORWARD and BACKWARD, and the resulting requirements and constraints

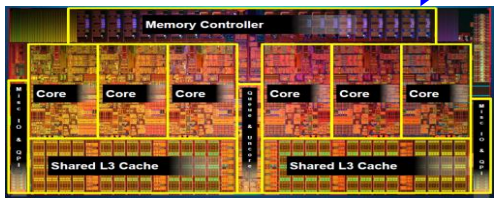


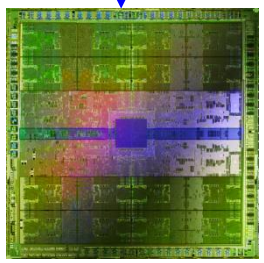
Heterogeneous Processors and Accelerators

Hybrid Main Memory



Persistent Memory/Storage





General Purpose GPUs

Every component and its interfaces, as well as entire system designs are being re-examined

Computer Architecture Today (IV)

- You can revolutionize the way computers are built, if you understand both the hardware and the software (and change each accordingly)

- You can invent new paradigms for computation, communication, and storage

… but, first …

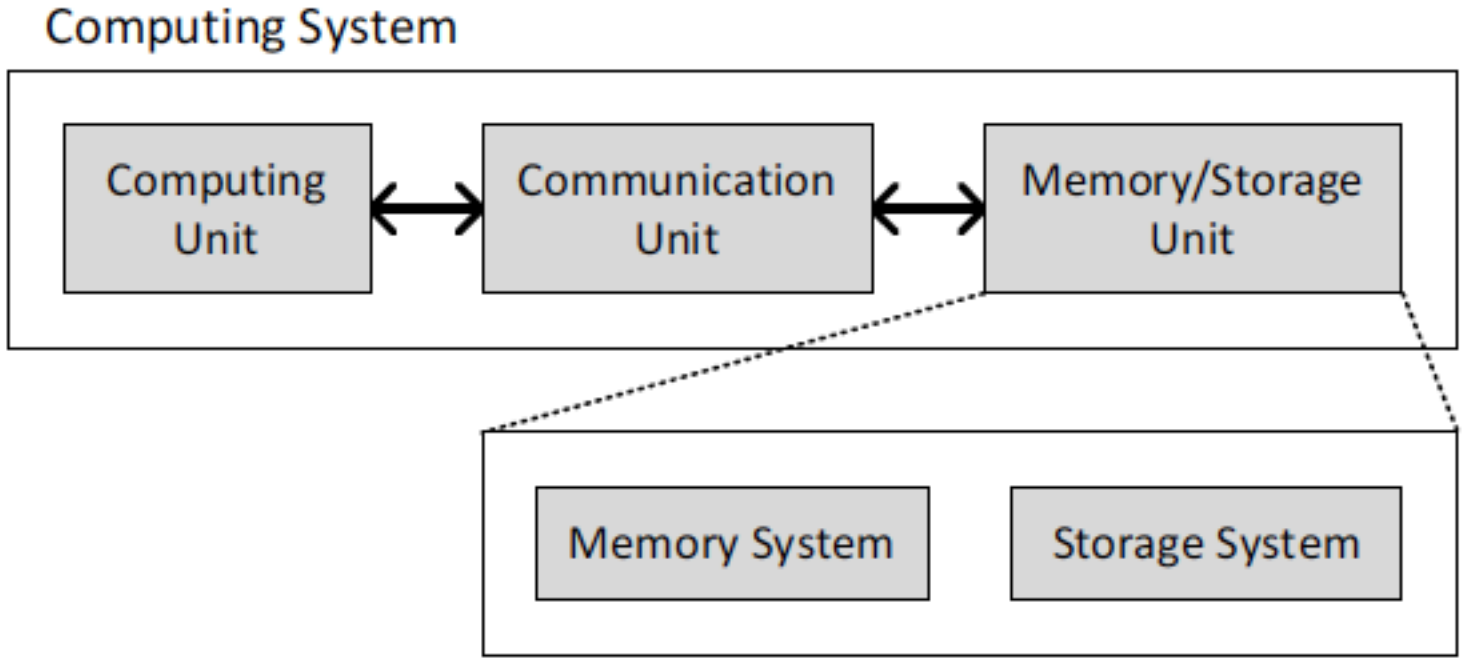
- Let’s understand the fundamentals…

- You can change the world only if you understand it well enough…
  - Especially the past and present dominant paradigms
  - And, their advantages and shortcomings – tradeoffs
  - And, what remains fundamental across generations
  - And, what techniques you can use and develop to solve problems

What is A Computer?

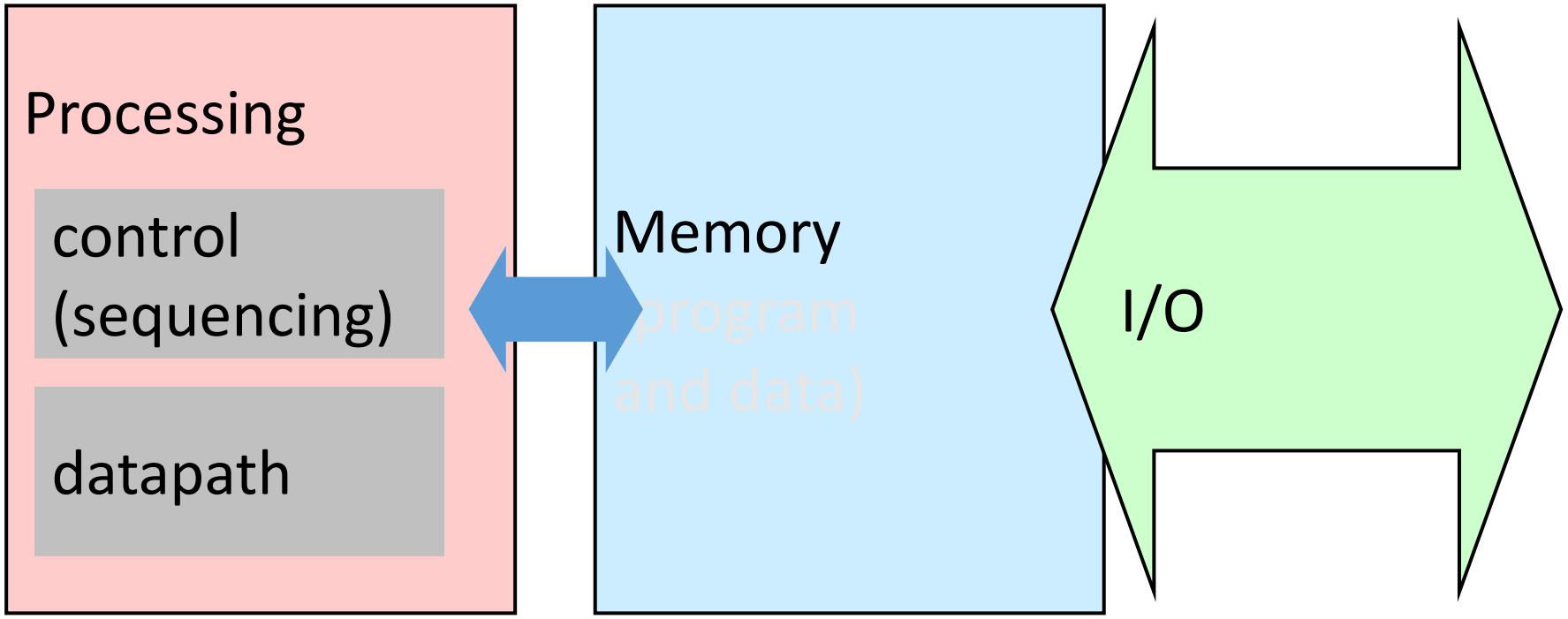
- Three key components

- Computation
- Communication
- Storage (memory)

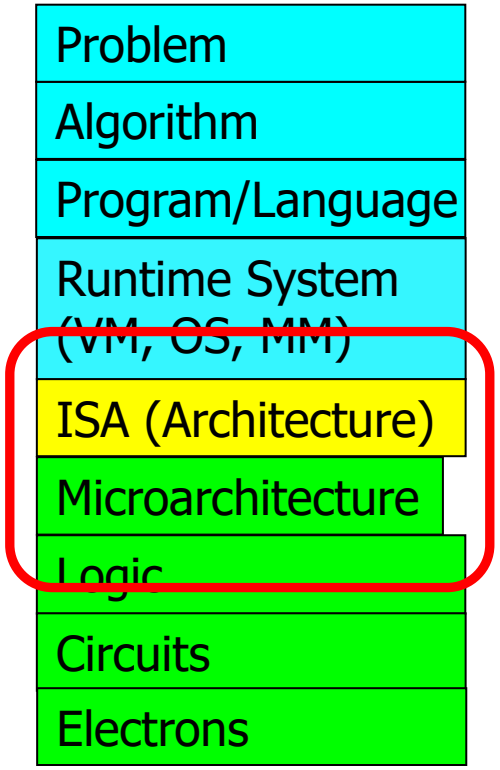


What is A Computer?

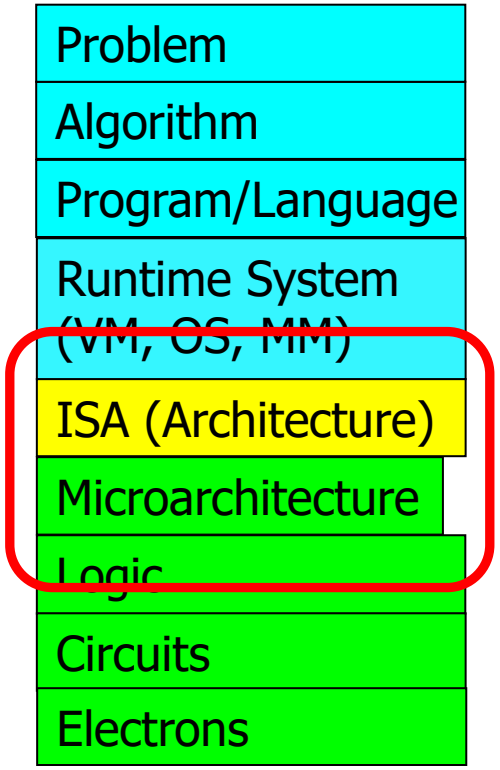
- We will cover all three components



Computer Architecture in Levels of Transformation



Computer Architecture in Levels of Transformation

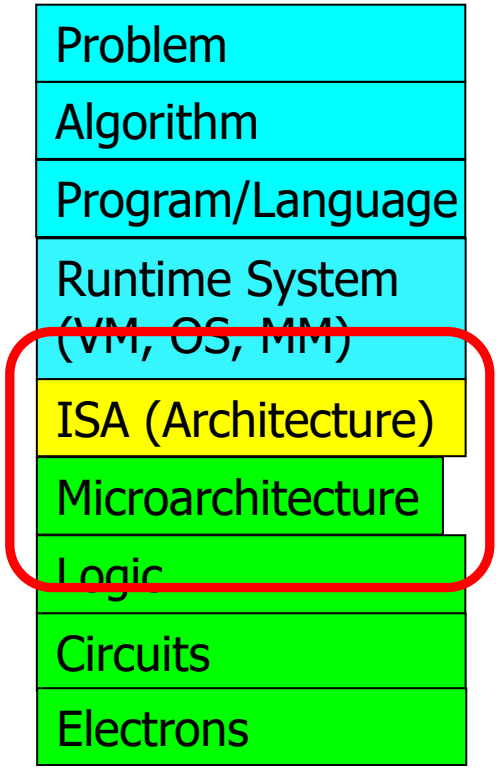


Sequential Logic and ROM

Combinational Logic

Bits and Logic Gates

Computer Architecture in Levels of Transformation



Procedures and Stacks

Compilers

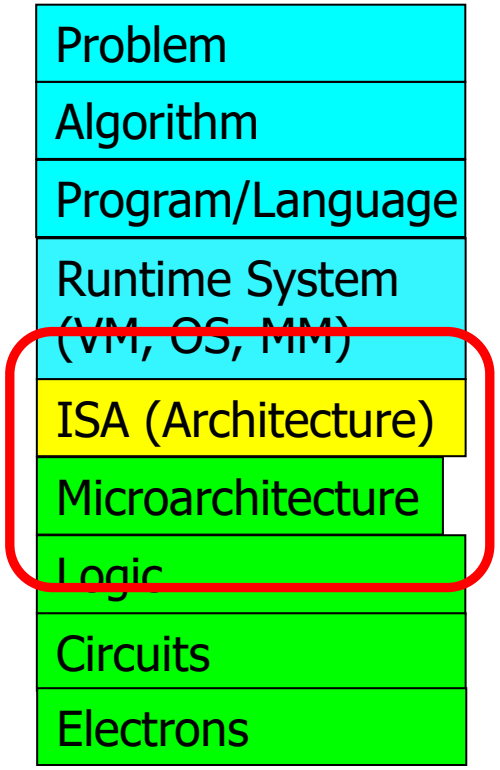
Programmable Machines (von Neumann, Data Flow)

Sequential Logic and ROM

Combinational Logic

Bits and Logic Gates

Computer Architecture in Levels of Transformation



Multiprocessors and Cache Coherence

Concurrency and Synchronization

Devices and Interrupts

Virtual Memory

Pipelining

Caches and Memory Hierarchy

Procedures and Stacks

Compilers

Programmable Machines (von Neumann, Data Flow)

Sequential Logic and ROM

Combinational Logic

Bits and Logic Gates

Digital Representations

Readings

- *David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)*
  - Chapter 2

Digital Abstraction

- **Encoding Information using Voltages**
  - **Representing analog information with voltages**

Digital Abstraction

- **Encoding Information using Voltages**
  - **Representing analog information with voltages**

Digital Abstraction

- **Encoding Information using Voltages**
  - **Representing analog information with voltages**

- **720 X 405 image**
- **Assigning different voltages for different colors**

Digital Abstraction

- **Encoding Information using Voltages**
  - **Representing analog information with voltages**

- **720 X 405 image**
- **Assigning different voltages for different colors**
- **291600 Dots -> maybe more than 100000 different voltages needed**

Digital Abstraction

- **Encoding Information using Voltages**
  - **Representing analog information with voltages**

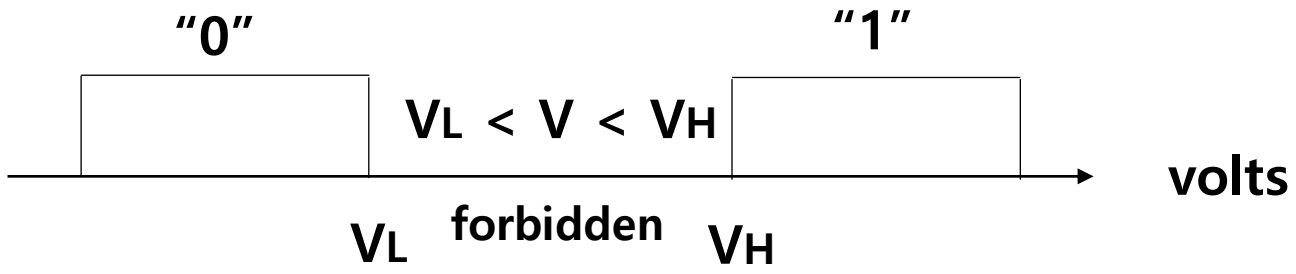
- **720 X 405 image**
- **Assigning different voltages for different colors**
- **291600 Dots -> maybe more than 100000 different voltages needed**
- **Would Fail because of noise**

Digital Abstraction

- Using Voltages Digitally -> “0” and “1”

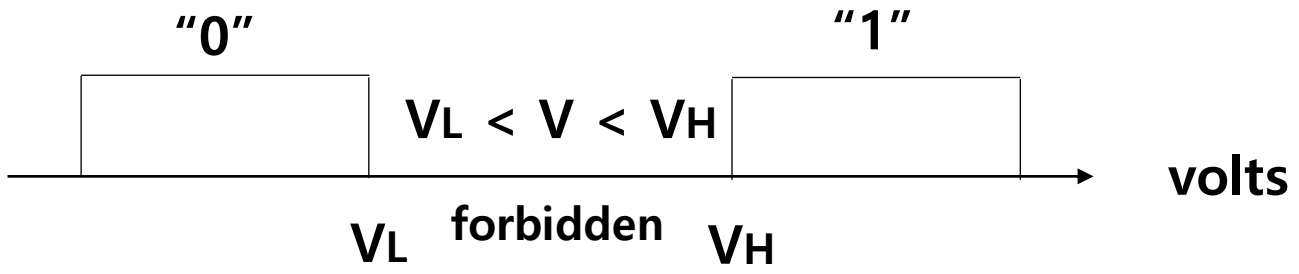
Digital Abstraction

- Using Voltages Digitally -> “0” and “1”



Digital Abstraction

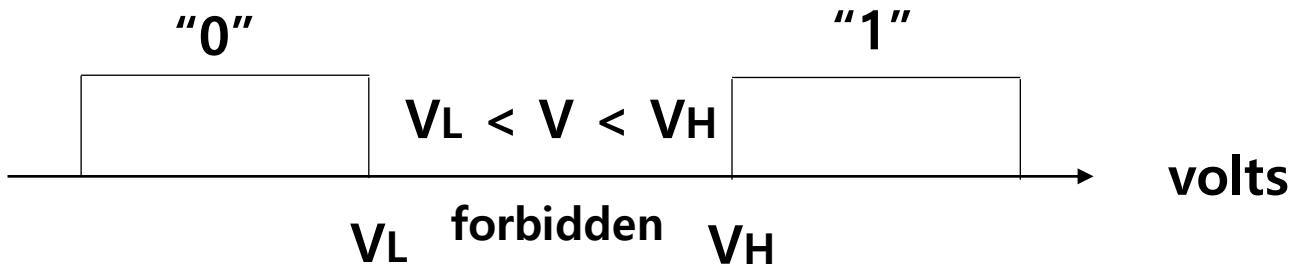
- Using Voltages Digitally -> “0” and “1”



- 720 X 405 image
- 291600 Dots -> Assign each dot with 8 bits

Digital Abstraction

- Using Voltages Digitally -> “0” and “1”



- 720 X 405 image
- 291600 Dots -> Assign each dot with 8 bits
- 2,332,800 bits
- Small amount of voltages

Combinational logic

Computer Architecture

Combinational Digital Systems

- **Digital Processing Element**

Functional Spec: Truth Table

Timing Spec: t_{PD}

Combinational Digital Systems

- **Digital Processing Element**

Functional Spec: Truth Table

Timing Spec: t_{PD}

- **Combinational Digital Systems**
  - **Each circuit element is combinational**

Combinational Digital Systems

- **Digital Processing Element**

Functional Spec: Truth Table

Timing Spec: t_{PD}

- **Combinational Digital Systems**
  - **Each circuit element is combinational**
  - **Every input is connected to one output or constant “0”s and “1”s**

Combinational Digital Systems

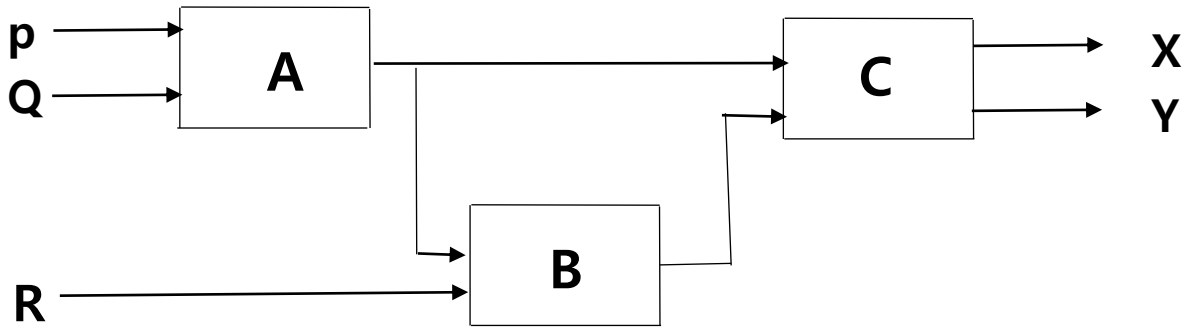
- **Digital Processing Element**

Functional Spec: Truth Table

Timing Spec: t_{PD}

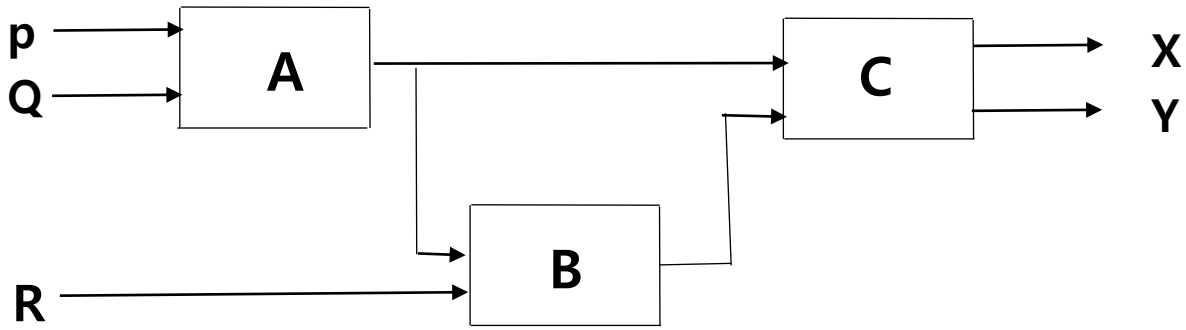
- **Combinational Digital Systems**
  - **Each circuit element is combinational**
  - **Every input is connected to one output or constant “0”s and “1”s**
  - **No directed cycles**

Combinational Digital Systems



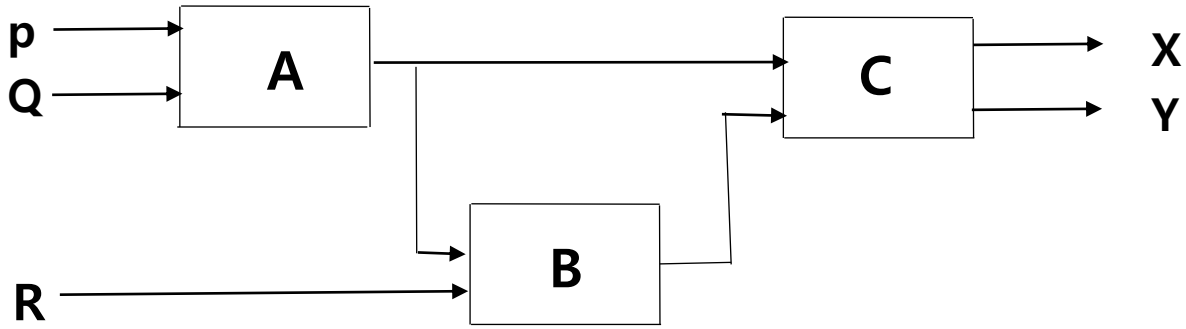
- Is this combinational?

Combinational Digital Systems



- Is this combinational?
  - If P, Q, R are digital inputs
  - If A, B, C are combinational

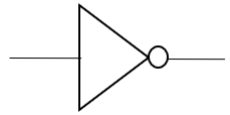
Combinational Digital Systems



- Is this combinational?
  - If P, Q, R are digital inputs
  - If A, B, C are combinational
    - Functional spec can be derived
    - Timing spec can be derived
    - No directed cycles inside A, B, C

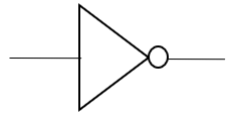
Digital Processing Element: Amplification

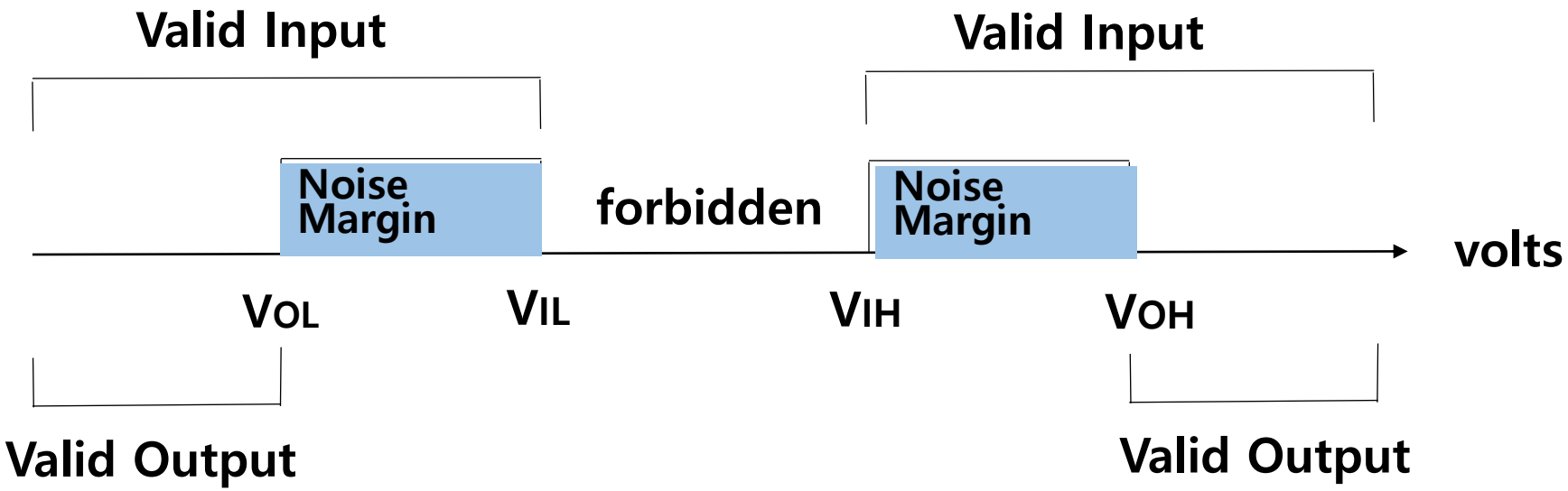
- Noise can change voltages slightly



Digital Processing Element: Amplification

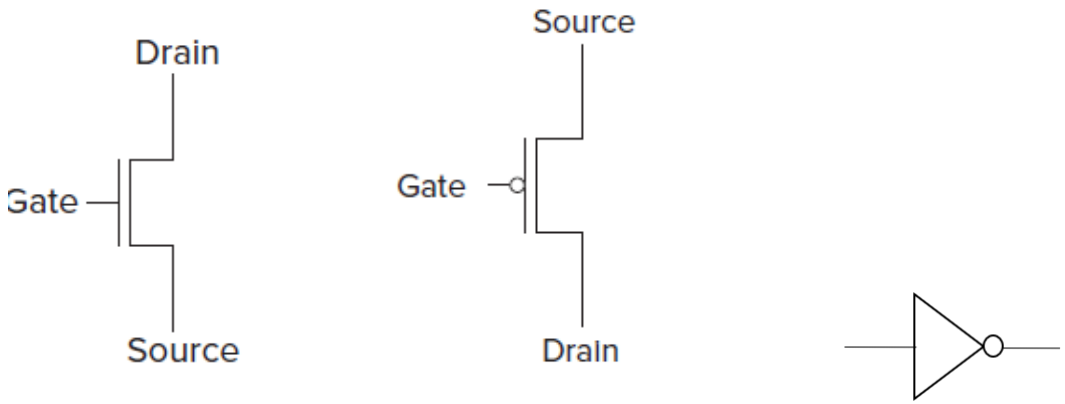
- Noise can change voltages slightly





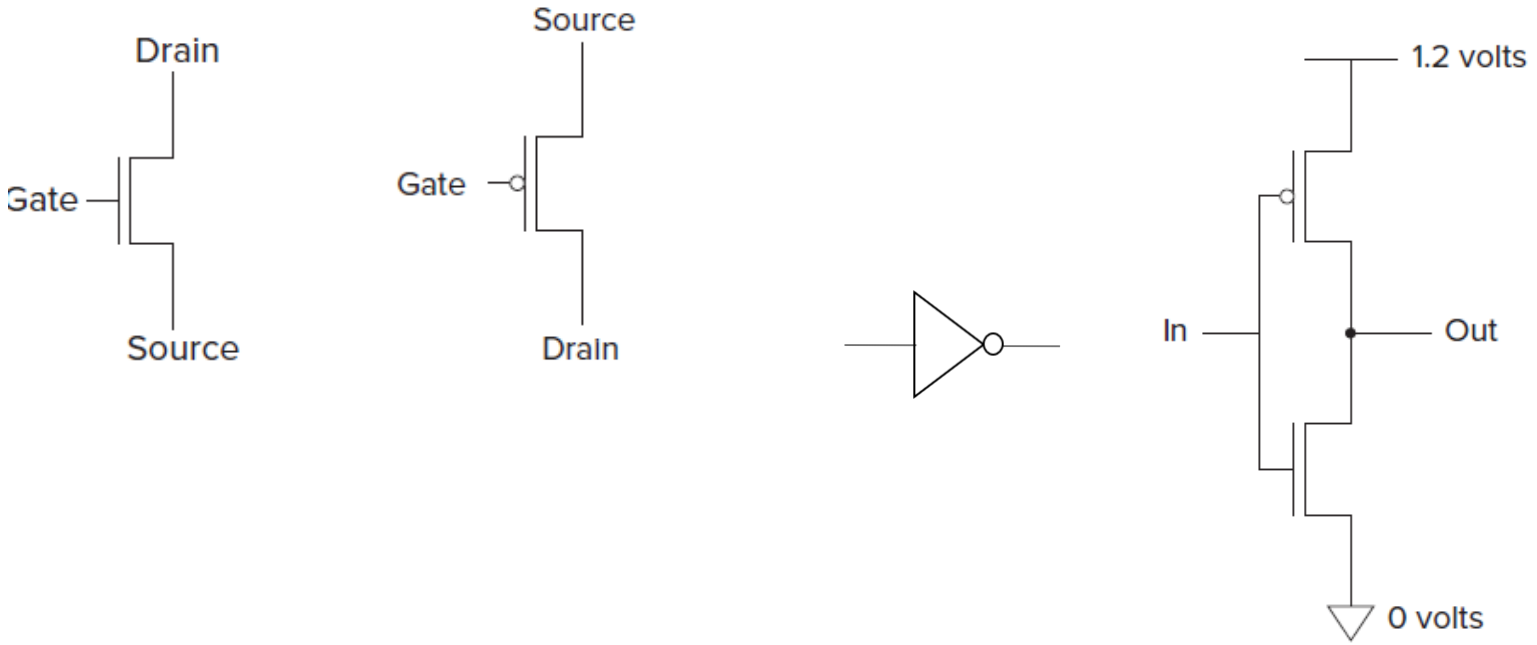
Transistors

- **Logic gates are composed of transistors**



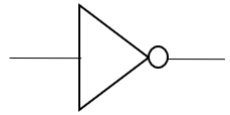
Transistors

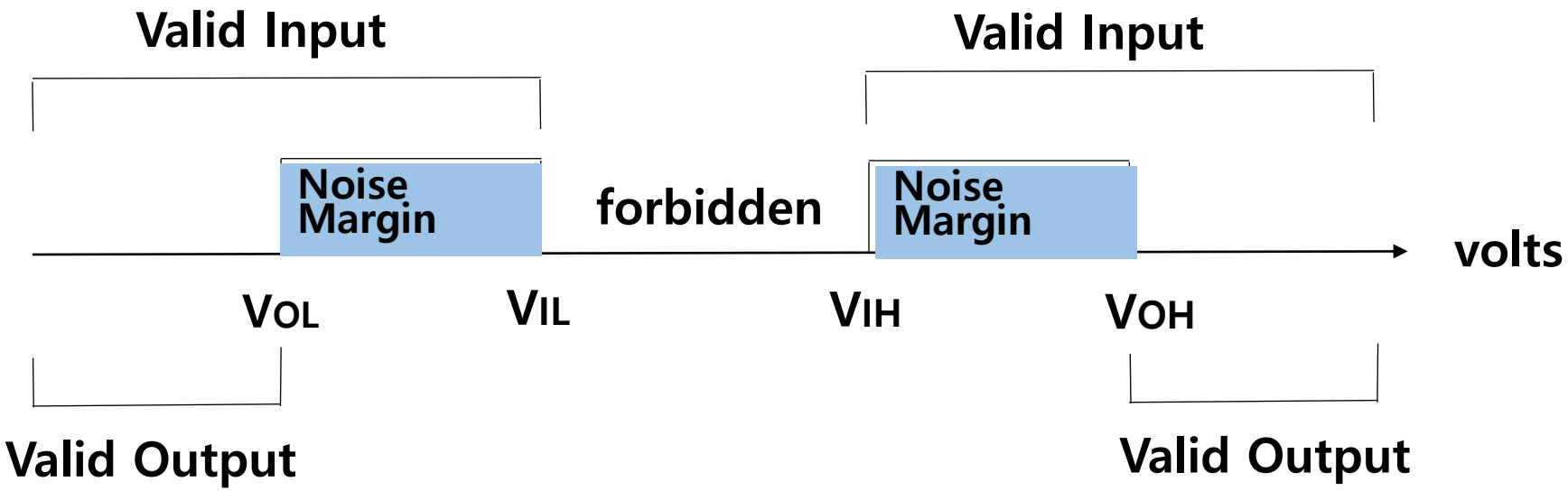
- **Logic gates are composed of transistors**



Digital Processing Element: Amplification

- Noise can change voltages slightly





- Silicon has wide noise margins

Functional Specifications

- Functional spec can be represented by
  - Truth tables

- Combinational logic

- Boolean expressions (Sum Of Products)

Functional Specifications

- Functional spec can be represented by
  - Truth tables

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Combinational logic

  - Boolean expressions (Sum Of Products)

Functional Specifications

- Functional spec can be represented by
  - Truth tables

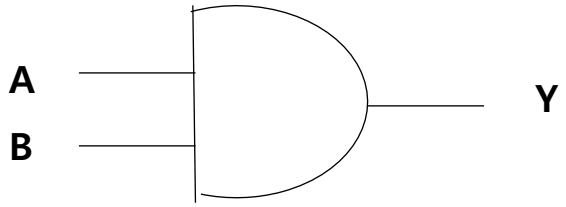
C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Combinational logic

- Boolean expressions (Sum Of Products)
  - $Y = \bar{C}\bar{B}A + \bar{C}B\bar{A} + C\bar{B}\bar{A} + CBA$

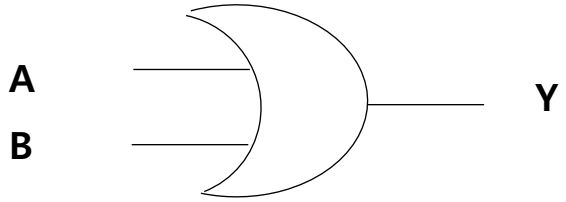
Building blocks for SOP

- **AND**



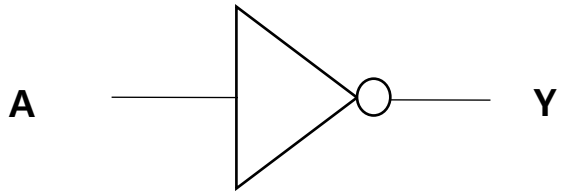
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

- **OR**



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

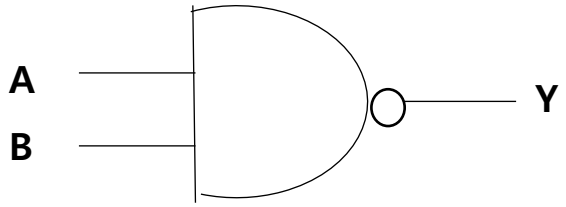
- **Inverter**



A	Y
0	1
1	0

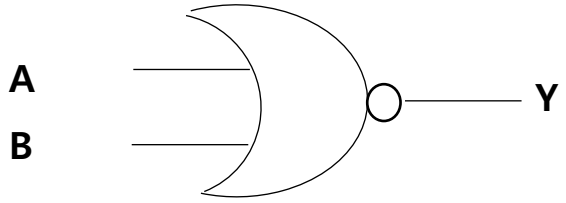
More building blocks for SOP

- **NAND**



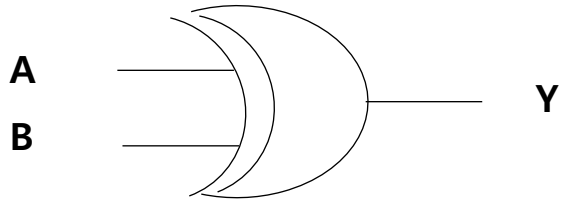
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

- **NOR**



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

- **XOR**



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Logic simplification

- **OR rules** $a+1=1,\ \ a+0=a,\ a+a=a$
- **AND rules** $a1=a,\ a0=0,\ aa=a$

Logic simplification

- OR rules — a+1=1, a+0=a, a+a=a
- AND rules — a1=a, a0=0, aa=a
- Commutative — a+b=b+a, ab=ba
- Associative — (a+b)+c=a+(b+c), (ab)c=a(bc)
- Distributive — a(b+c)=ab+ac, a+bc=?

Logic simplification

- OR rules $a+1=1,\ \ a+0=a,\ a+a=a$
- AND rules $a1=a,\ a0=0,\ aa=a$
- Commutative $a+b=b+a,\ ab=ba$
- Associative $(a+b)+c=a+(b+c),\ (ab)c=a(bc)$
- Distributive $a(b+c)=ab+ac,\ a+bc=(a+b)(a+c)$

Logic simplification

- OR rules — $a+1=1,\ \ a+0=a,\ a+a=a$
- AND rules — $a1=a,\ a0=0,\ aa=a$
- Commutative — $a+b=b+a,\ ab=ba$
- Associative — $(a+b)+c=a+(b+c),\ (ab)c=a(bc)$
- Distributive — $a(b+c)=ab+ac,\ a+bc=(a+b)(a+c)$
- Complements — $a+\bar{a}=1,\ a\bar{a}=0$

Logic simplification

- OR rules — $a+1=1,\ \ a+0=a,\ a+a=a$
- AND rules — $a1=a,\ a0=0,\ aa=a$
- Commutative — $a+b=b+a,\ ab=ba$
- Associative — $(a+b)+c=a+(b+c),\ (ab)c=a(bc)$
- Distributive — $a(b+c)=ab+ac,\ a+bc=(a+b)(a+c)$
- Complements — $a+\bar{a}=1,\ a\bar{a}=0$
- Absorption — $a+ab=?,\ a+\bar{a}b=?,\ a(a+b)=?,\ a(\bar{a}+b)=?$

Logic simplification

- OR rules $a+1=1,\ \ a+0=a,\ a+a=a$
- AND rules $a1=a,\ a0=0,\ aa=a$
- Commutative $a+b=b+a,\ ab=ba$
- Associative $(a+b)+c=a+(b+c),\ (ab)c=a(bc)$
- Distributive $a(b+c)=ab+ac,\ a+bc=(a+b)(a+c)$
- Complements $a+\bar{a}=1,\ a\bar{a}=0$
- Absorption $a+ab=a,\ a+\bar{a}b=a+b,\ a(a+b)=a,\ a(\bar{a}+b)=ab$

Logic simplification

- OR rules — $a+1=1,\ \ a+0=a,\ a+a=a$
- AND rules — $a1=a,\ a0=0,\ aa=a$
- Commutative — $a+b=b+a,\ ab=ba$
- Associative — $(a+b)+c=a+(b+c),\ (ab)c=a(bc)$
- Distributive — $a(b+c)=ab+ac,\ a+bc=(a+b)(a+c)$
- Complements — $a+\bar{a}=1,\ a\bar{a}=0$
- Absorption — $a+ab=a,\ a+\bar{a}b=a+b,\ a(a+b)=a,\ a(\bar{a}+b)=ab$
- Reduction — $ab+\bar{a}b=?,\ (a+b)(\bar{a}+b)=?$

Logic simplification

- OR rules $a+1=1,\ \ a+0=a,\ a+a=a$
- AND rules $a1=a,\ a0=0,\ aa=a$
- Commutative $a+b=b+a,\ ab=ba$
- Associative $(a+b)+c=a+(b+c),\ (ab)c=a(bc)$
- Distributive $a(b+c)=ab+ac,\ a+bc=(a+b)(a+c)$
- Complements $a+\bar{a}=1,\ a\bar{a}=0$
- Absorption $a+ab=a,\ a+\bar{a}b=a+b,\ a(a+b)=a,\ a(\bar{a}+b)=ab$
- Reduction $ab+\bar{a}b=b,\ (a+b)(\bar{a}+b)=b$

Logic simplification

- OR rules — $a+1=1,\ a+0=a,\ a+a=a$
- AND rules — $a1=a,\ a0=0,\ aa=a$
- Commutative — $a+b=b+a,\ ab=ba$
- Associative — $(a+b)+c=a+(b+c),\ (ab)c=a(bc)$
- Distributive — $a(b+c)=ab+ac,\ a+bc=(a+b)(a+c)$
- Complements — $a+\bar{a}=1,\ a\bar{a}=0$
- Absorption — $a+ab=a,\ a+\bar{a}b=a+b,\ a(a+b)=a,\ a(\bar{a}+b)=ab$
- Reduction — $ab+\bar{a}b=b,\ (a+b)(\bar{a}+b)=b$
- DeMorgan's Law — $\overline{ab}=?,\ \overline{a+b}=?$

Logic simplification

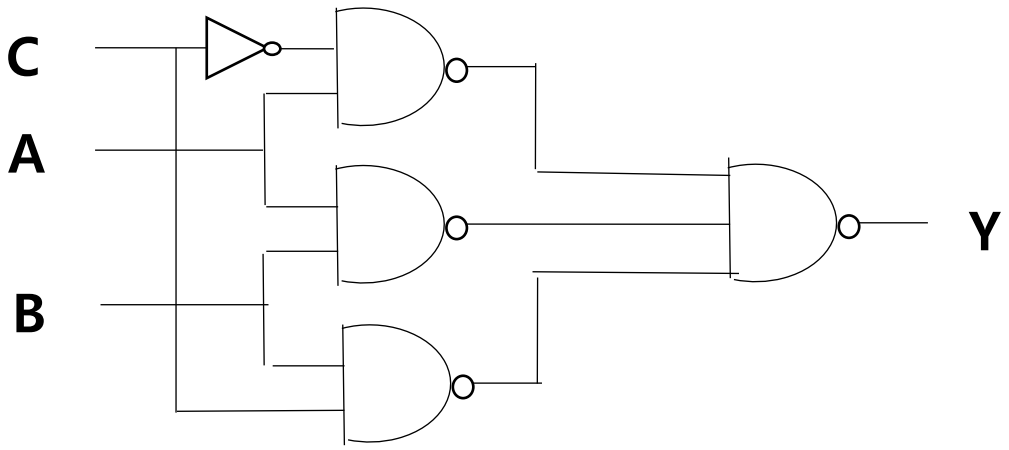
- OR rules $a+1=1,\ \ a+0=a,\ a+a=a$
- AND rules $a1=a,\ a0=0,\ aa=a$
- Commutative $a+b=b+a,\ ab=ba$
- Associative $(a+b)+c=a+(b+c),\ (ab)c=a(bc)$
- Distributive $a(b+c)=ab+ac,\ a+bc=(a+b)(a+c)$
- Complements $a+\bar{a}=1,\ a\bar{a}=0$
- Absorption $a+ab=a,\ a+\bar{a}b=a+b,\ a(a+b)=a,\ a(\bar{a}+b)=ab$
- Reduction $ab+\bar{a}b=b,\ (a+b)(\bar{a}+b)=b$
- DeMorgan's Law $\overline{ab}=\bar{a}+\bar{b},\ \overline{a+b}=\bar{a}\bar{b}$

SOP with NANDs, NORs, Inverters

- $Y = A\overline{C} + AB + BC$

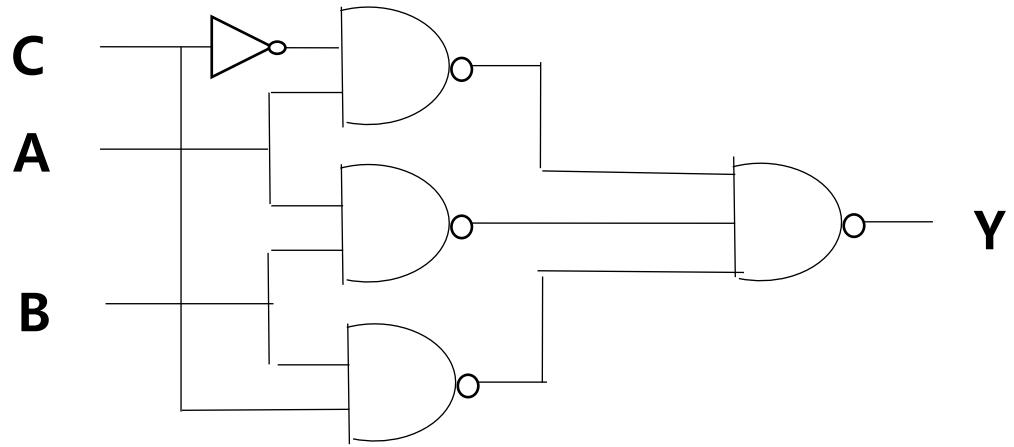
SOP with NANDs, NORs, Inverters

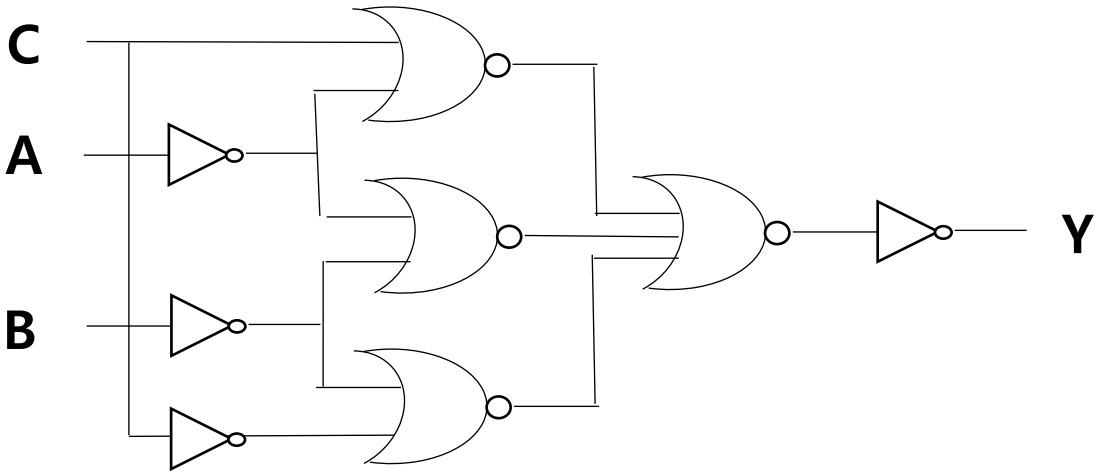
- $Y = A\overline{C} + AB + BC$



SOP with NANDs, NORs, Inverters

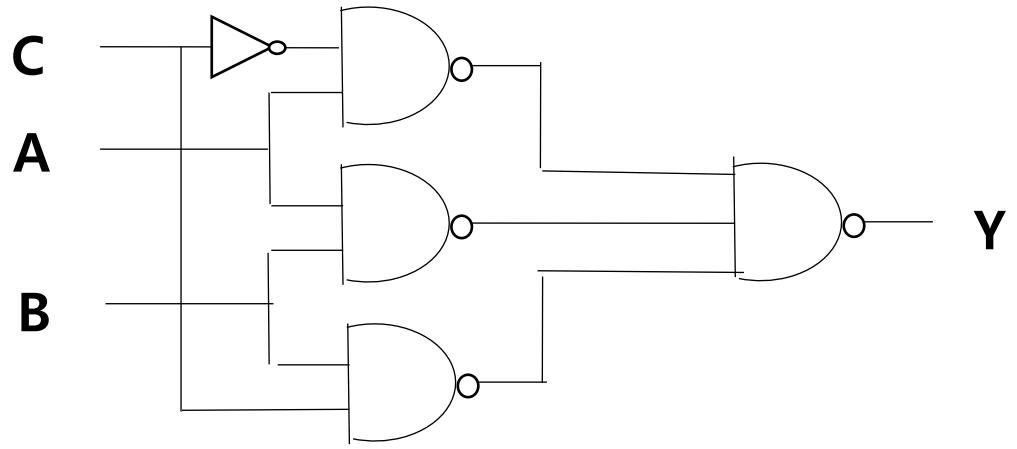
- $Y = A\overline{C} + AB + BC$

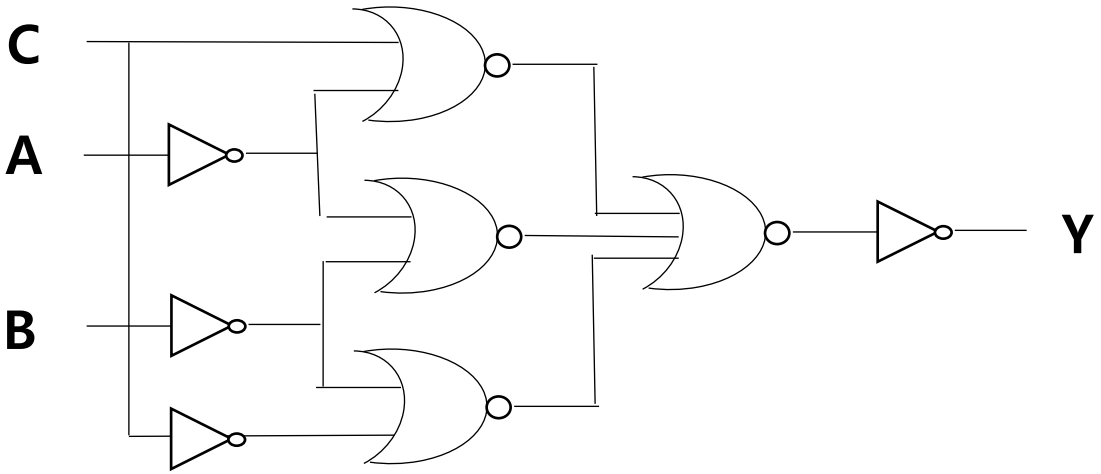




SOP with NANDs, NORs, Inverters

- $Y = A\overline{C} + AB + BC$

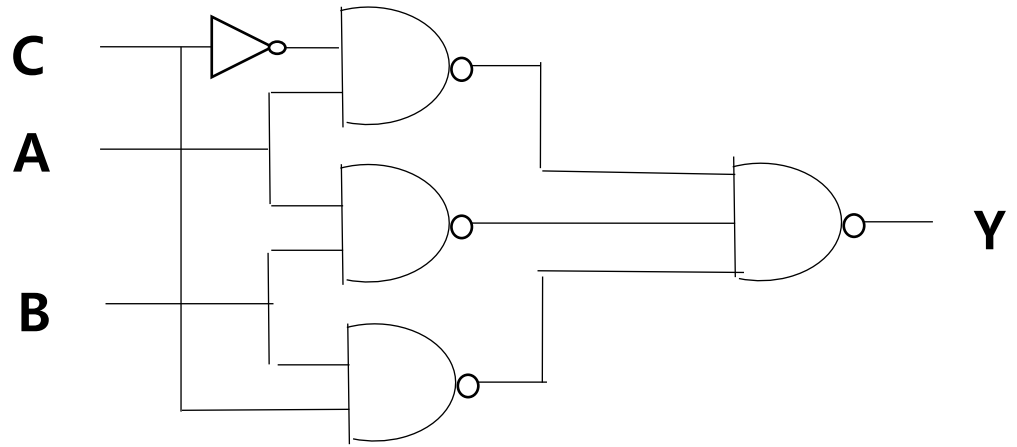


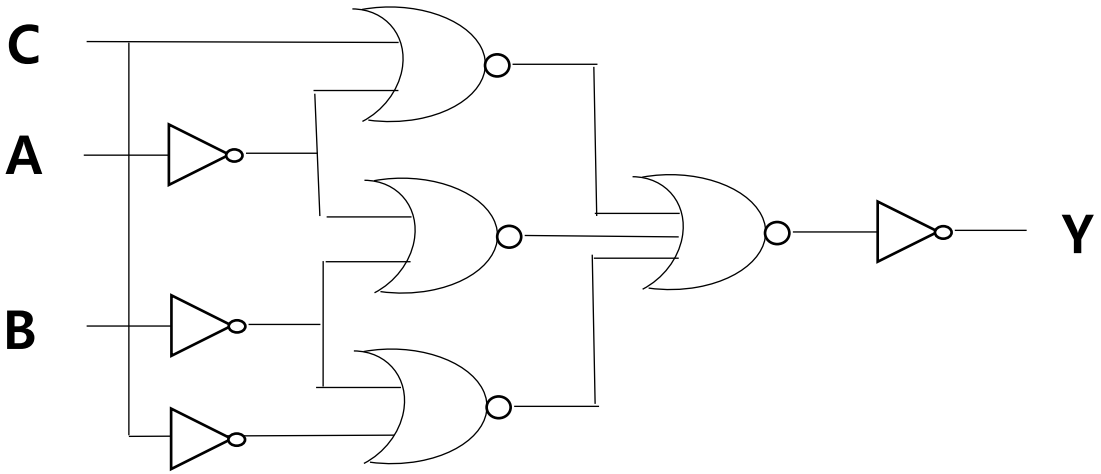


- NAND, NOR gates are faster than AND, OR gates

SOP with NANDs, NORs, Inverters

- $Y = A\overline{C} + AB + BC$





- NAND, NOR gates are faster than AND, OR gates
  - Property of CMOS (complementary metal-oxide-semiconductor)

Boolean minimization

- **Truth table**

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- **Boolean expressions (Sum Of Products)**

Boolean minimization

- **Truth table**

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- **Boolean expressions (Sum Of Products)**
  - $Y = \bar{C}\bar{B}A + \bar{C}BA + CB\bar{A} + CBA$

Boolean minimization

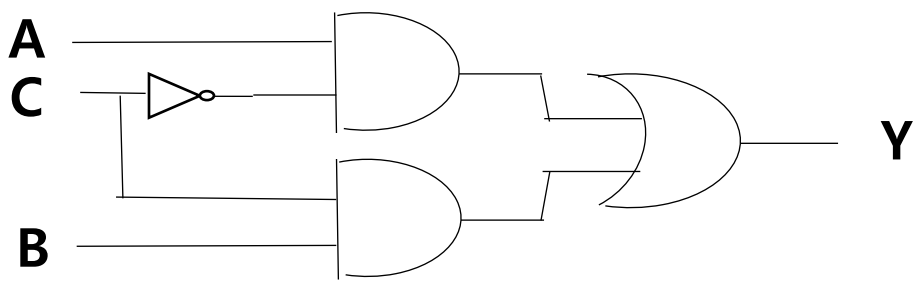
- **Truth table**

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- **Boolean expressions (Sum Of Products)**
  - $Y = \bar{C}\bar{B}A + \bar{C}BA + CB\bar{A} + CBA$
    $= \bar{C}A + CB$

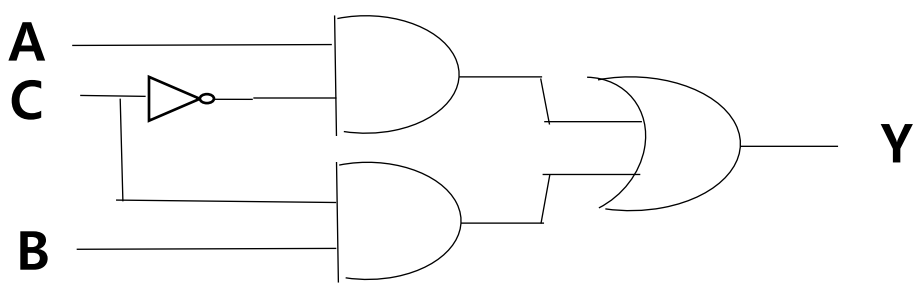
Case for non-minimal SOP

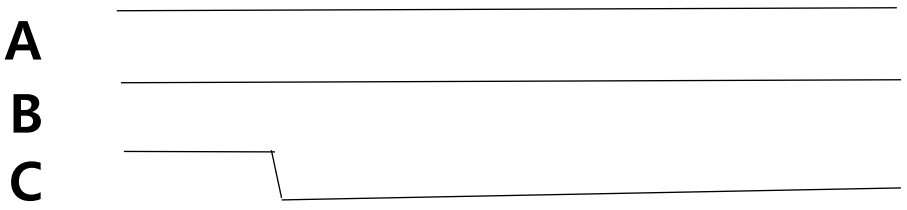
- $Y = \bar{C}A + CB$



Case for non-minimal SOP

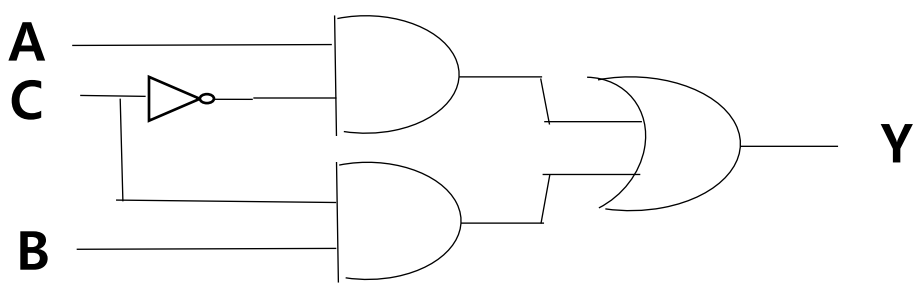
- $Y = \bar{C}A + CB$

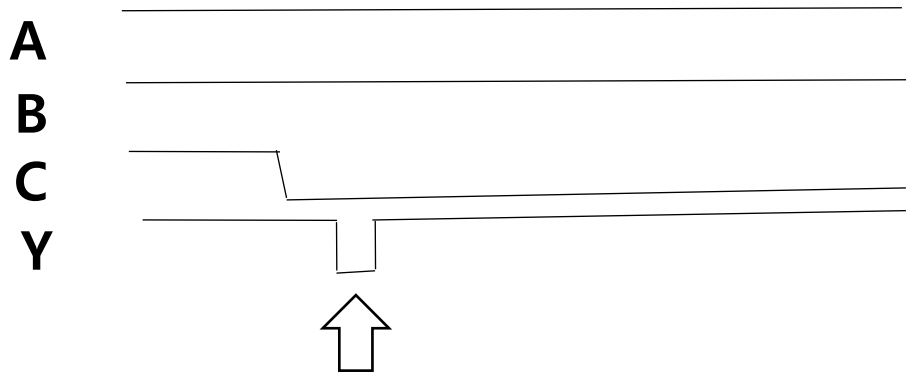




Case for non-minimal SOP

- $Y = \bar{C}A + CB$

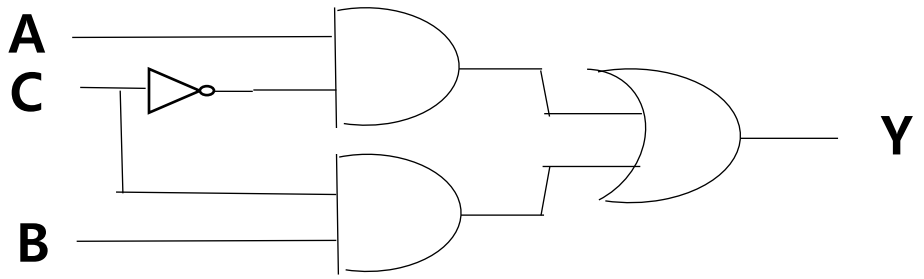


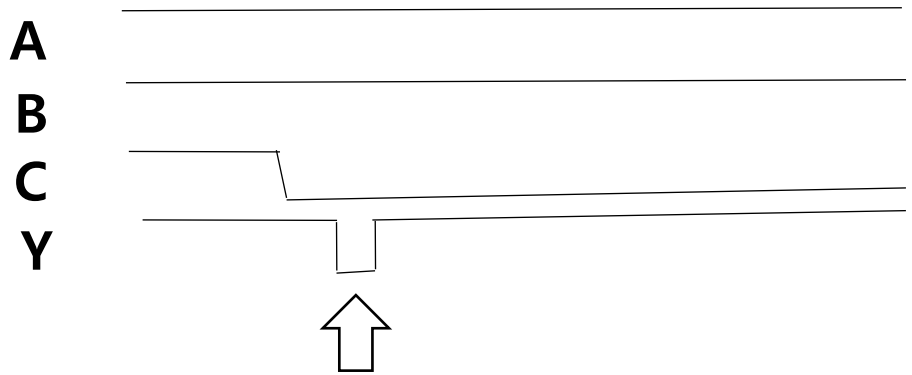


Glitch

Case for non-minimal SOP

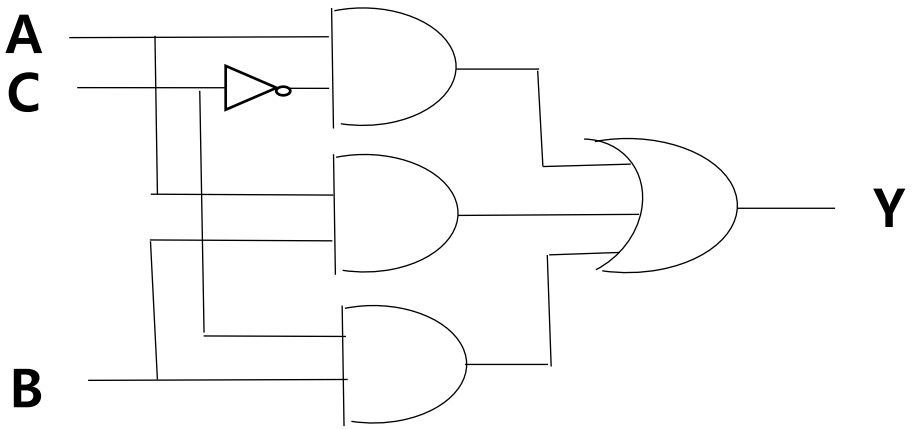
- $Y = \bar{C}A + CB$





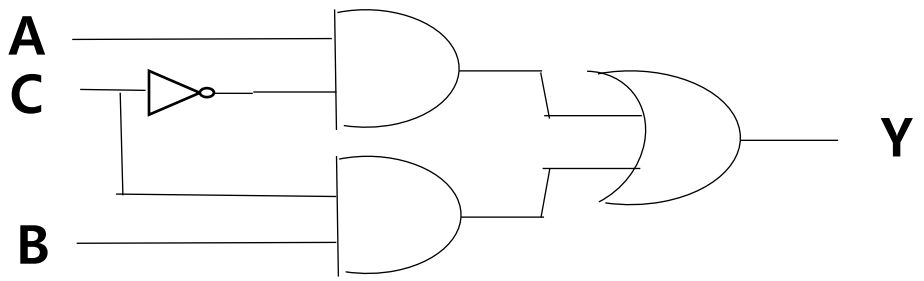
Glitch

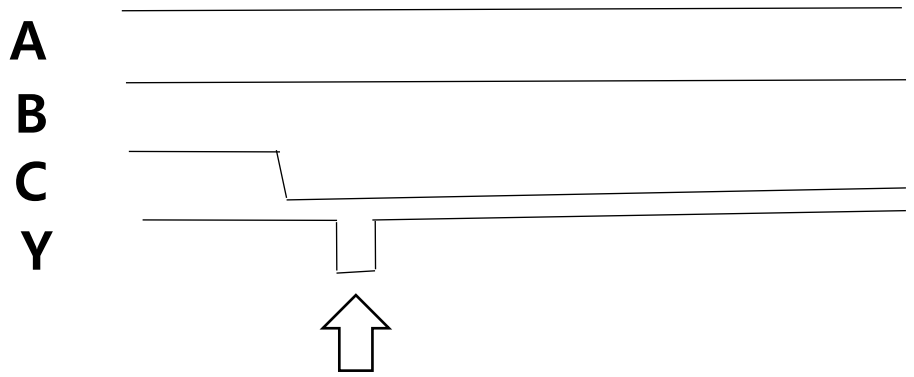
- $Y = \bar{C}A + AB + CB$



Case for non-minimal SOP

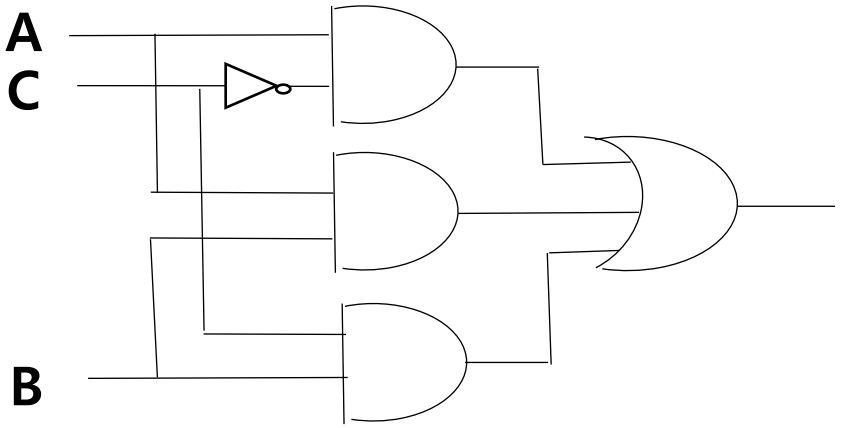
- $Y = \bar{C}A + CB$





Glitch

- $Y = \bar{C}A + AB + CB$



Lenient

Leniency guaranteed minimization

- **Truth table**

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Leniency guaranteed minimization

- **Truth table**

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- **Karnaugh maps**
  - **Representing adjacencies geometrically**

Leniency guaranteed minimization

- **Truth table**

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- **Karnaugh maps**
  - **Representing adjacencies geometrically**

C:BA	00	01	11	10
0	0	1	1	0
1	0	0	1	1

Finding prime implicants on K-map

- **Prime implicants**

Finding prime implicants on K-map

- **Prime implicants**
  - **Rectangular region of the K-map with value 1**

Finding prime implicants on K-map

- **Prime implicants**
  - **Rectangular region of the K-map with value 1**
  - **Start from 2**n, 2**(n-1), … , 2**

Finding prime implicants on K-map

- **Prime implicants**
  - **Rectangular region of the K-map with value 1**
  - **Start from 2**n, 2**(n-1), … , 2**
  - **Can overlap other implicants**

Finding prime implicants on K-map

- **Prime implicants**
  - **Rectangular region of the K-map with value 1**
  - **Start from 2**n, 2**(n-1), … , 2**
  - **Can overlap other implicants**
  - **A prime implicant is not completely contained in any implicants**

Finding prime implicants on K-map

- **Prime implicants**
  - **Rectangular region of the K-map with value 1**
  - **Start from 2**n, 2**(n-1), … , 2**
  - **Can overlap other implicants**
  - **A prime implicant is not completely contained in any implicants**

C:BA	00	01	11	10
0	1	1	1	1
1	1	1	1	1

Finding prime implicants on K-map

- **Prime implicants**
  - **Rectangular region of the K-map with value 1**
  - **Start from 2**n, 2**(n-1), … , 2**
  - **Can overlap other implicants**
  - **A prime implicant is not completely contained in any implicants**

C:BA	00	01	11	10
0	1	1	1	1
1	1	1	1	1

C:BA	00	01	11	10
0	0	1	1	0
1	1	1	1	1

Finding prime implicants on K-map

- **Prime implicants**
  - **Rectangular region of the K-map with value 1**
  - **Start from 2**n, 2**(n-1), … , 2**
  - **Can overlap other implicants**
  - **A prime implicant is not completely contained in any implicants**

C:BA	00	01	11	10
0	1	1	1	1
1	1	1	1	1

C:BA	00	01	11	10
0	0	1	1	0
1	1	1	1	1

C:BA	00	01	11	10
0	1	0	1	1
1	1	0	1	1

Finding prime implicants on K-map

- **Prime implicants**
  - **Rectangular region of the K-map with value 1**
  - **Start from 2**n, 2**(n-1), … , 2**
  - **Can overlap other implicants**
  - **A prime implicant is not completely contained in any implicants**

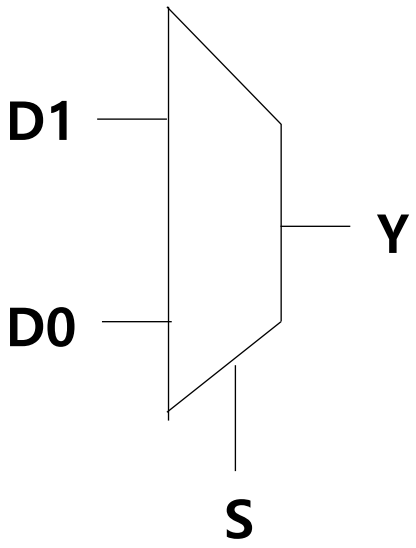
C:BA	00	01	11	10
0	1	1	1	1
1	1	1	1	1

C:BA	00	01	11	10
0	0	1	1	0
1	1	1	1	1

C:BA	00	01	11	10
0	1	0	1	1
1	1	0	1	1

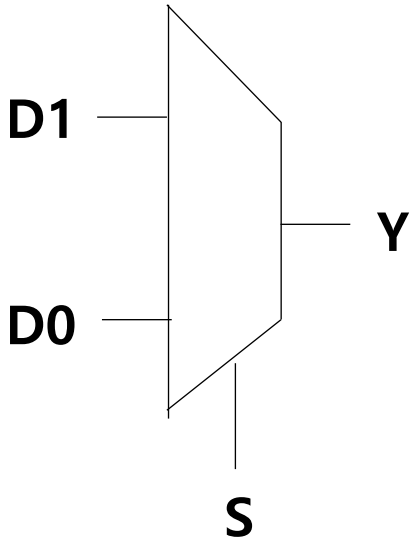
C:BA	00	01	11	10
0	0	0	1	1
1	1	0	1	1

Multiplexers



- Truth table

Multiplexers

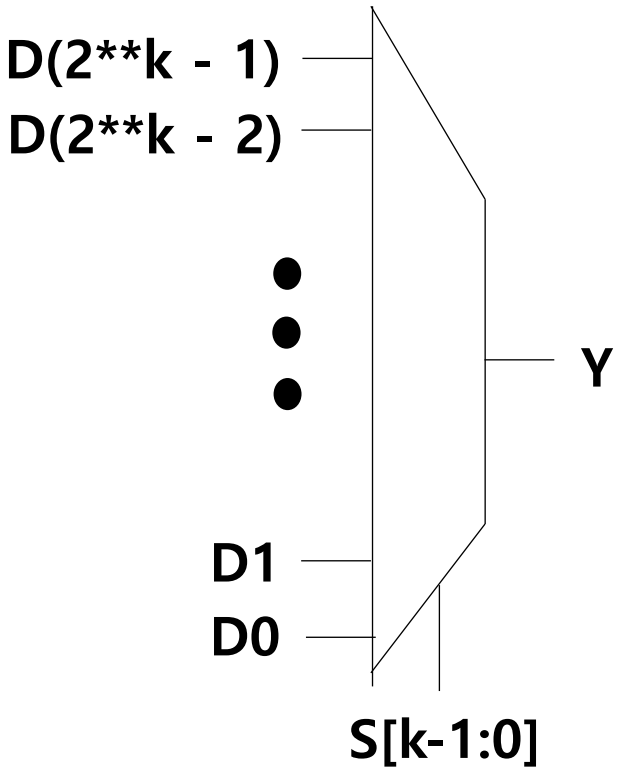


- **Truth table**

S	D1	D0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Multiplexers

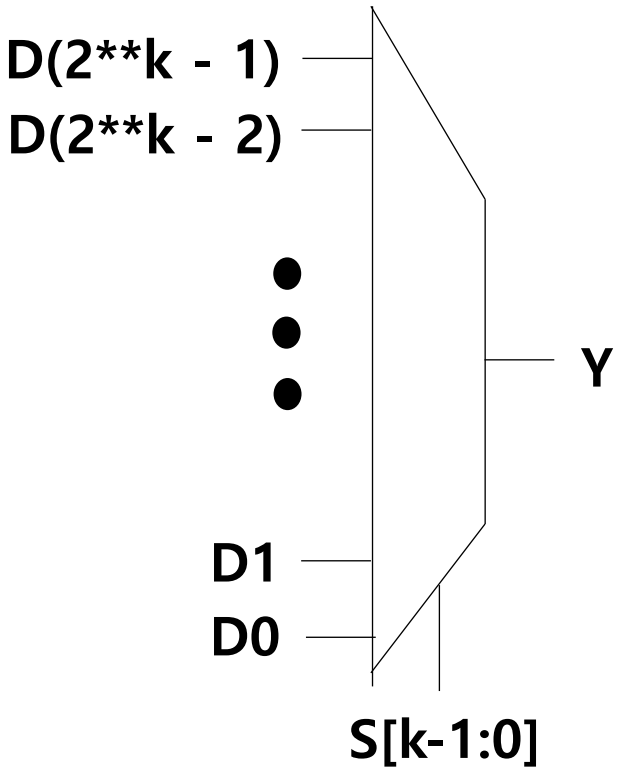
- MUXes can be generalized to 2**k data inputs and k select inputs



S	D1	D0	Y
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**1**

Multiplexers

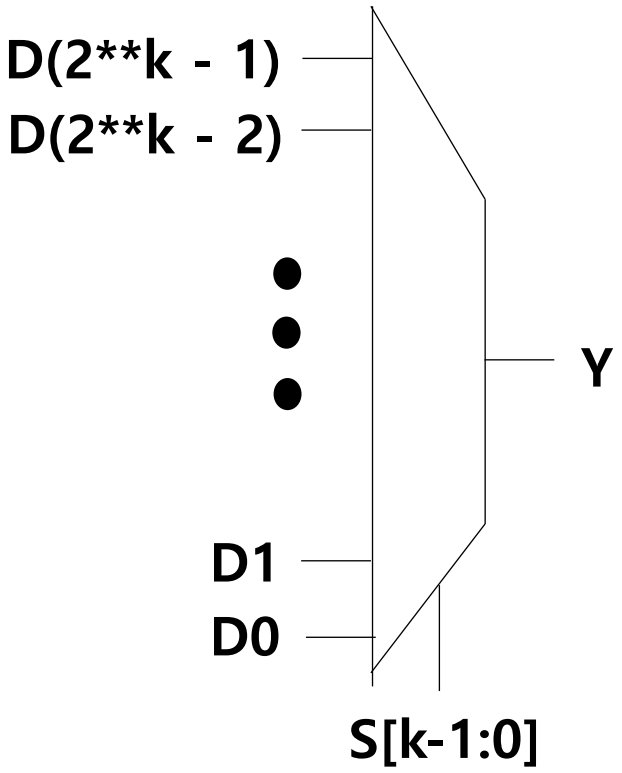
- MUXes can be generalized to 2**k data inputs and k select inputs
  - Implementing some arbitrary Boolean functions using ONLY MUXes



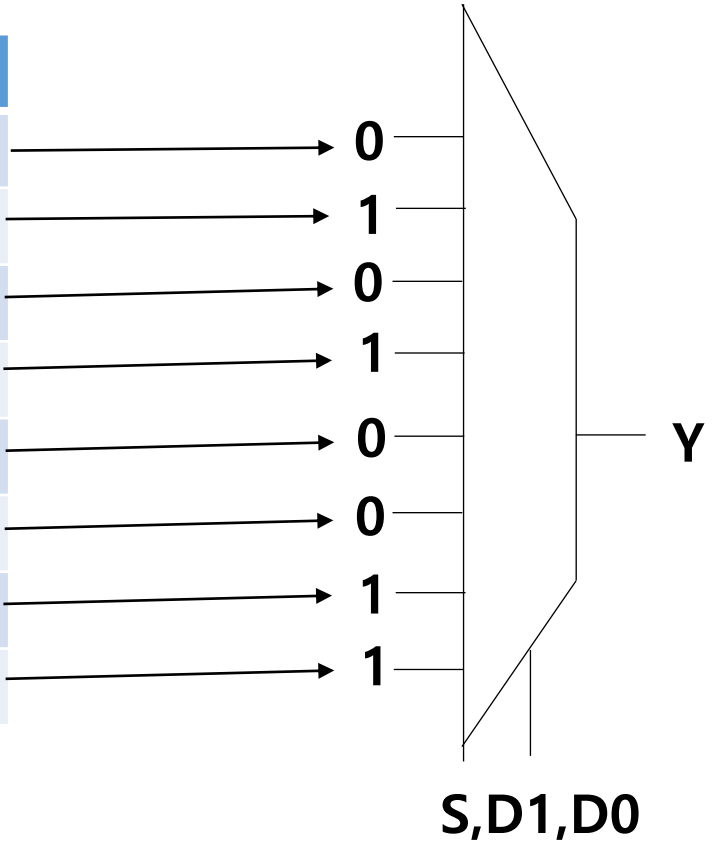
S	D1	D0	Y
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**1**

Multiplexers

- MUXes can be generalized to 2**k data inputs and k select inputs
  - Implementing some arbitrary Boolean functions using ONLY MUXes

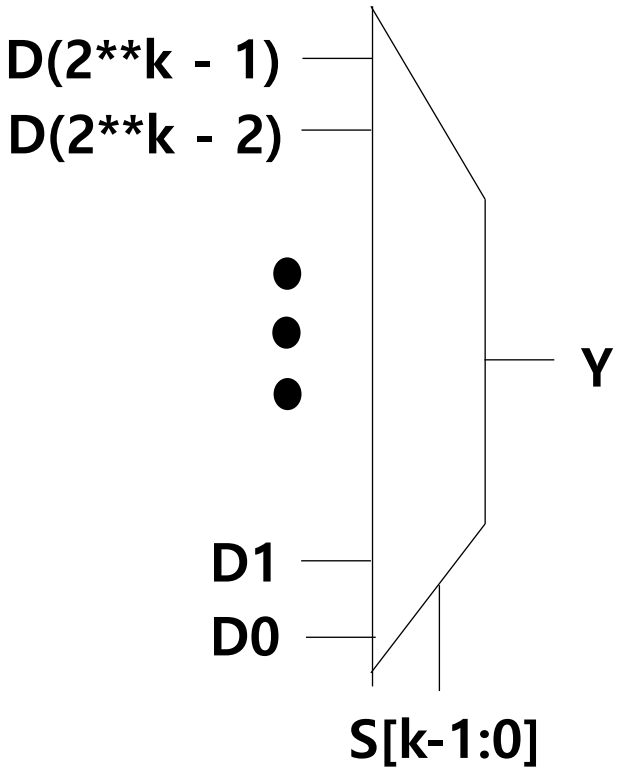


S	D1	D0	Y
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**1**

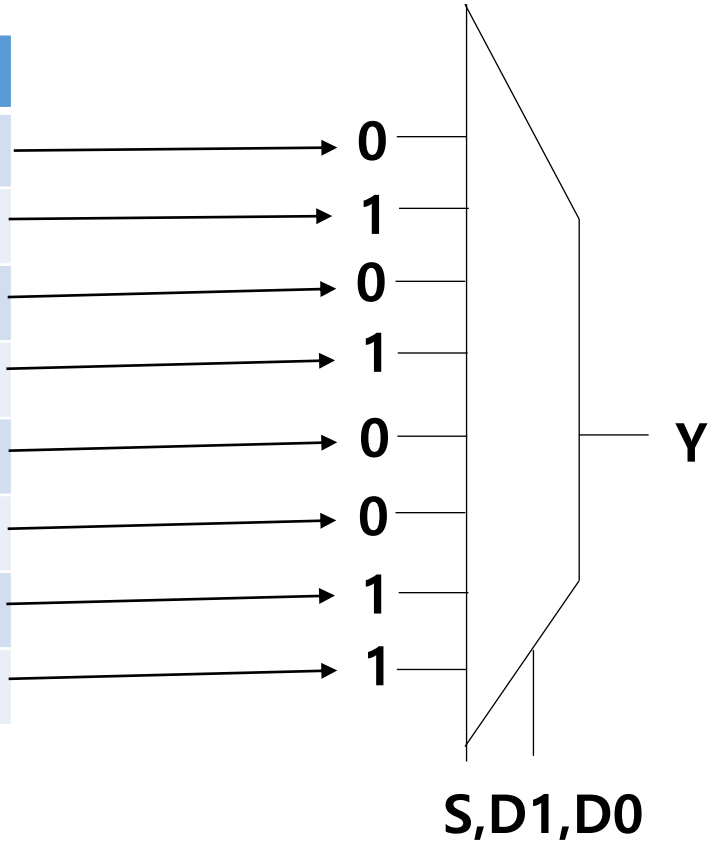


Multiplexers

- MUXes can be generalized to 2**k data inputs and k select inputs
  - Implementing some arbitrary Boolean functions using ONLY MUXes
  - Good for prototyping but not fast

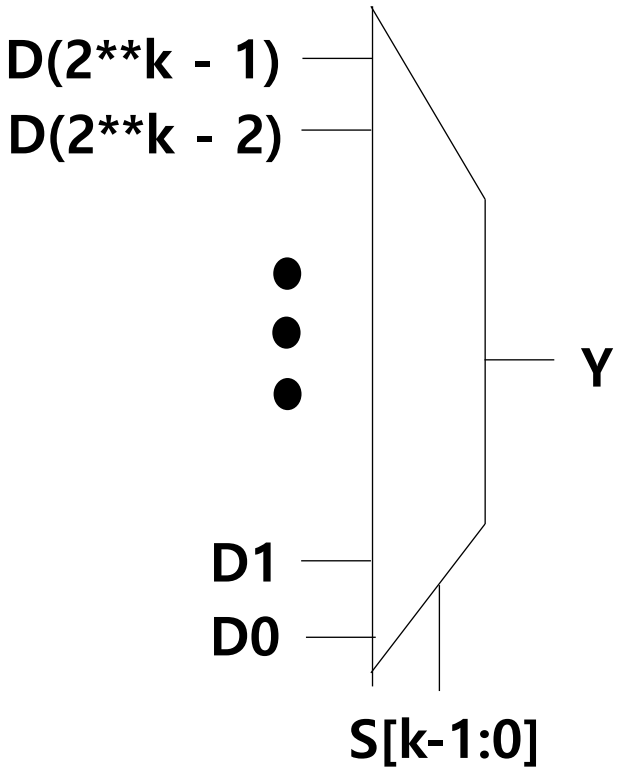


S	D1	D0	Y
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**1**

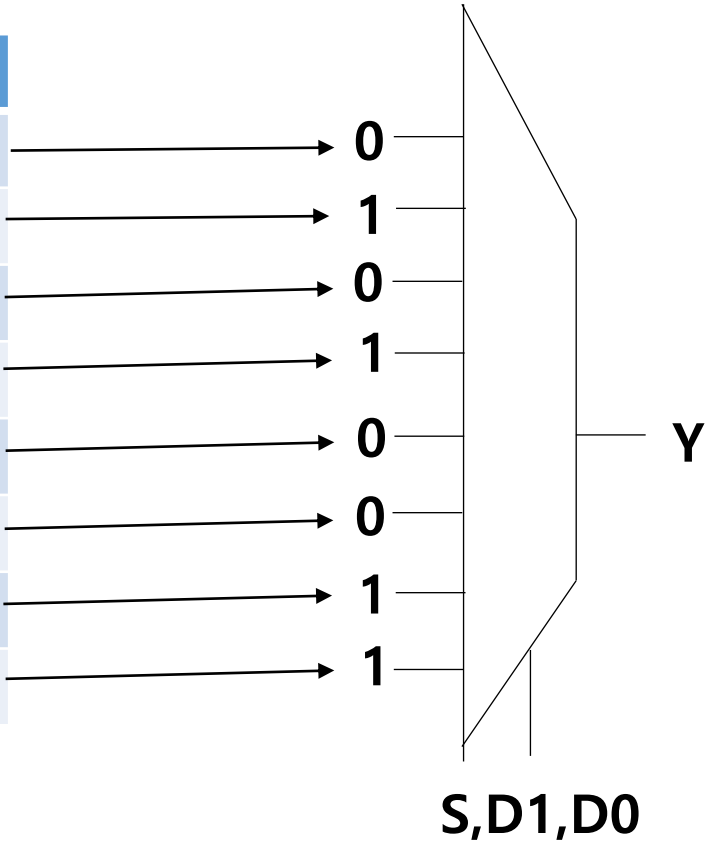


Multiplexers

- MUXes can be generalized to 2**k data inputs and k select inputs
  - Implementing some arbitrary Boolean functions using ONLY MUXes
  - Good for prototyping but not fast
  - Not practical if k is big

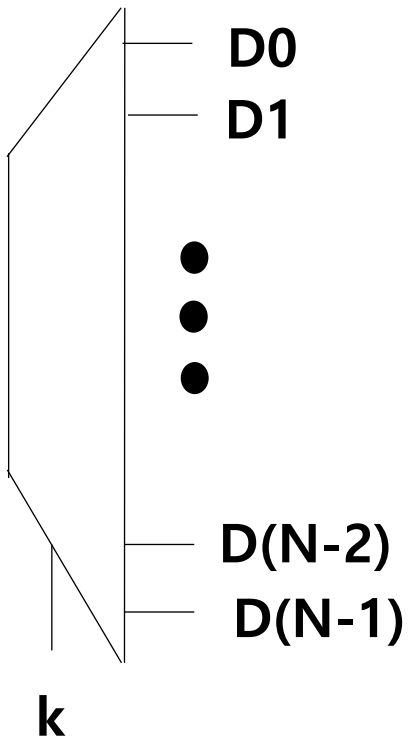


S	D1	D0	Y
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**1**



Decoders and ROM

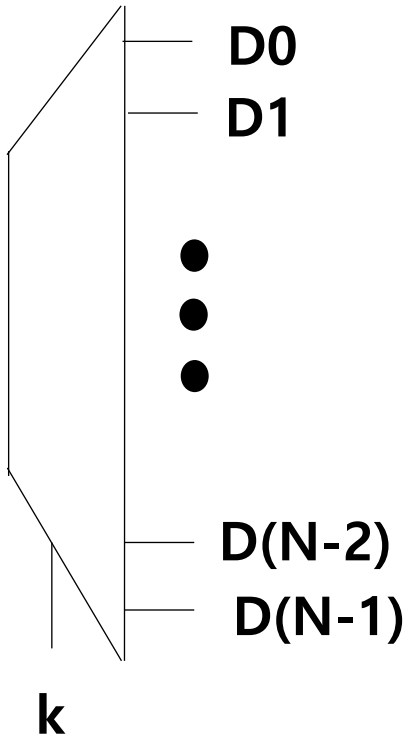
- Decoder
  - k select inputs
  - N=2**k outputs
  - One of Ds is HIGH



- Read Only Memory (ROM)

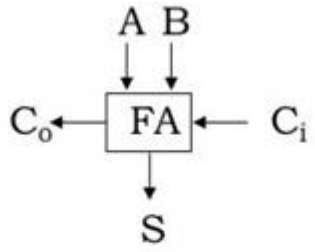
Decoders and ROM

- Decoder
  - k select inputs
  - N=2**k outputs
  - One of Ds is HIGH



- Read Only Memory (ROM)

Full Adder



A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1